

Experiment-3-OPEN
HANDED

AIM: Simulate cross correlation and autocorrelation on discrete time signals.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from pydub import AudioSegment
from scipy.signal import correlate # FFT-based fast correlation

def mp3_to_np(filepath, max_samples=None):
    audio = AudioSegment.from_file(filepath)
    audio = audio.set_channels(1) # convert to mono
    audio = audio.set_frame_rate(8000) # downsample to 8 kHz
    samples = np.array(audio.get_array_of_samples())

    if max_samples is not None:
        samples = samples[:max_samples] # take only required part
    return samples

# Paths to your songs
song1_path = r'Vande Mataram (Maa Tujhe Salaam)-(Mr-Jat.in).mp3'
song2_path = r'Vande Mataram Karaoke -HQ.mp3'

# fix max samples (e.g. 50,000 samples ~ 6 sec of audio at 8kHz)
MAX_SAMPLES = 50000

# Read limited audio samples
data1 = mp3_to_np(song1_path, MAX_SAMPLES)
data2 = mp3_to_np(song2_path, MAX_SAMPLES)

# Ensure both same length
min_len = min(len(data1), len(data2))
signal1 = data1[:min_len]
signal2 = data2[:min_len]

# Cross-correlation (FFT method = fast)
cross_corr = correlate(signal1, signal2, mode='full', method='fft')
# Autocorrelation
auto_corr = correlate(signal1, signal1, mode='full', method='fft')

# Print (optional)
print("Cross-correlation length:", len(cross_corr))
print("Autocorrelation length:", len(auto_corr))

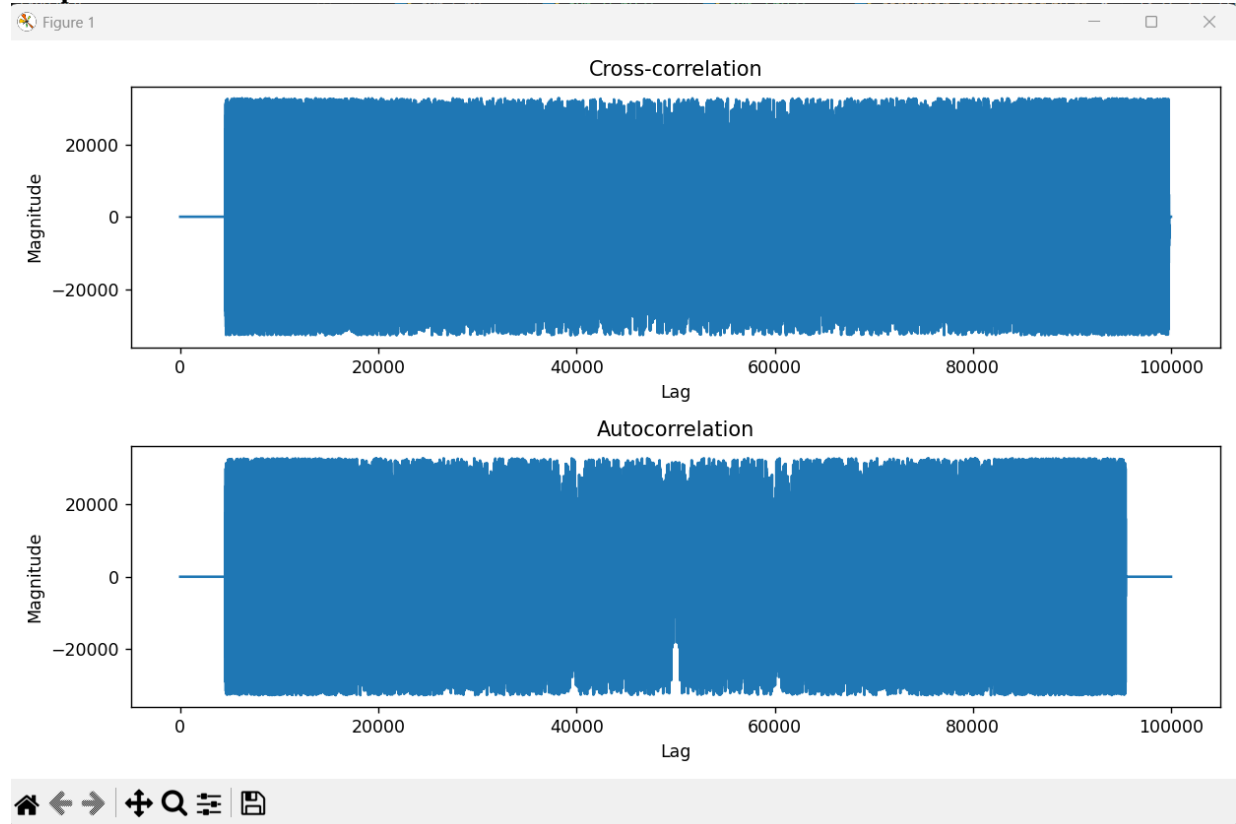
# Plot
plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1)
plt.plot(cross_corr)
plt.title('Cross-correlation')
plt.xlabel('Lag')
plt.ylabel('Magnitude')
```

```
plt.subplot(2, 1, 2)  
plt.plot(auto_corr)  
plt.title('Autocorrelation')  
plt.xlabel('Lag')  
plt.ylabel('Magnitude')
```

```
plt.tight_layout()  
plt.show()
```

Output:



Observation

- Cross-correlation: Measures similarity between original and karaoke versions, peak indicates time alignment
- Autocorrelation: Shows signal self-similarity, maximum peak at center (zero lag)
- Both signals show high correlation confirming they are versions of the same song
- Useful for audio synchronization, pattern detection, and similarity measurement